

Guía práctica de implementación experimental

TFG: comunicación PS-PL en ZedBoard mediante periférico AXI4-Lite

Documento de trabajo orientado a ejecutar la parte práctica del TFG sin modificar todavía la memoria.

Fecha	20-04-2026
Autor de la guía	ChatGPT (soporte técnico-académico para el TFG)
Base documental	TFG 20-04, petición inicial del TFG y documentación técnica oficial de AMD, Buildroot y Linux.
Entorno fijado	ZedBoard / Zynq-7000, Vivado 2022.2, Vitis 2022.2, Ubuntu 20.04 LTS, Buildroot a congelar como release exacta 2023.02.x.
Propósito	Definir tareas, dependencias, artefactos y criterios de cierre para ejecutar la implementación práctica y la validación reproducible descritas en la teoría.

Cómo debe usarse este documento

No es un capítulo de memoria, sino una hoja de ruta ejecutable. Cada tarea incluye: objetivo, dependencias, pasos, entregables, criterio de cierre, riesgos y evidencias a conservar para la futura redacción del TFG.

1. Marco fijo del trabajo

La versión 20-04 del TFG ya fija el alcance experimental realista: el caso de estudio principal no es una comparación exhaustiva de AXI4-Lite, AXI4-Full y AXI-Stream sobre hardware real, sino el diseño, integración y validación de un periférico AXI4-Lite accesible desde Linux en ZedBoard. AXI4-Full y AXI-Stream quedan como referencias arquitectónicas y bibliográficas. [TFG-20-04]

La petición inicial del TFG sí exigía Linux en ARM, diseño en FPGA, automatización de pruebas y recogida de datos. Esta guía convierte esa intención general en un flujo práctico acotado, defendible y reproducible. [PETICION]

Hechos fijados por la memoria

Plataforma ZedBoard / Zynq-7000; flujo Vivado 2022.2 -> Vitis 2022.2 -> Buildroot -> device tree -> validación desde espacio de usuario; objetivo principal centrado en latencia, estabilidad y comportamiento de acceso a registros. [TFG-20-04]

Decisiones recomendadas para ejecutar la práctica

1) Empezar con un único periférico AXI4-Lite de control/monitorización. 2) Usar el camino PS master -> PL slave (GP) para el MVP. 3) Acceder desde Linux primero con /dev/mem y mmap. 4) Dejar UIO, interrupciones e instrumentación ILA como extensiones controladas.

Dudas que debes cerrar antes de tocar nada

La duda más importante no es hardware: es la versión exacta de Buildroot. En la memoria aparece "Buildroot 2023.2", pero el proyecto usa releases con formato YYYY.MM. Antes de empezar, congela si tu árbol real es 2023.02, 2023.02.x o un snapshot concreto. [BUILDROOT]

2. Arquitectura objetivo y límites del MVP

El diseño mínimo viable debe ser deliberadamente simple: Linux en el PS, acceso por M_AXI_GP0 a un periférico AXI4-Lite en la PL, reloj FCLK desde el PS, reset controlado y una ventana de registros suficientemente clara para poder medir accesos repetidos. La razón es metodológica: la memoria ya legitima que, en interfaces de control como AXI4-Lite, la métrica central sea la latencia y estabilidad del acceso a registros, no el throughput máximo de un camino de datos. [TFG-20-04] [UG585]

PS (ARM Cortex-A9, Linux)

-> M_AXI_GP0

-> AXI interconnect / SmartConnect

-> AXI4-Lite custom IP (PL)

-> registros de control, estado y contadores

Elemento	Decisión para el MVP	Qué queda fuera del MVP
Interfaz PS-PL	M_AXI_GP0 hacia un esclavo AXI4-Lite en la PL.	AXI HP / ACP / DMA / AXI-Stream no entran en la primera

Elemento	Decisión para el MVP	Qué queda fuera del MVP
		implementación.
Acceso software	Espacio de usuario mediante /dev/mem + mmap.	Driver kernel completo; UIO solo como ampliación.
Reloj PL	FCLK desde PS; fijar una frecuencia y documentarla.	Barridos de frecuencia salvo que queden muy bien acotados.
IP personalizado	Periférico sencillo de registros, contadores y eco de datos.	Acelerador complejo o datapath sostenido.
Métricas	Latencia media, dispersión, repetibilidad y consistencia funcional.	Benchmarking "completo" AXI4-Full/Stream sobre varias arquitecturas.

Zynq-7000 distingue claramente los puertos GP, adecuados para control de periféricos y acceso a registros, de los HP, orientados a mayor ancho de banda hacia DDR/OCM; esa separación encaja exactamente con el alcance elegido para el TFG. [UG585]

3. Flujo global que debe quedar implementado

Especificación del IP -> RTL + simulación -> Block Design -> bitstream + XSA
-> FSBL + BOOT.bin (Vitis/Bootgen)
-> Buildroot (U-Boot + kernel + DTB + rootfs)
-> SD/boot -> Linux -> /dev/mem -> binario de medida -> CSV -> análisis posterior

- Cada etapa debe dejar artefactos versionados: no avances si todavía dependes de archivos "sueltos" fuera del proyecto.
- No cambies dos variables de sistema a la vez. Primero haz que arranque. Después haz que lea un registro. Después automatiza. Después mide.
- La validación funcional precede a la medición. No tomes latencias de un periférico cuya semántica aún no está demostrada.
- La medida debe apoyarse en software ligero en espacio de usuario, pero el control de campaña puede seguir haciéndose con scripts Bash desde Linux o desde el host.

4. Tareas detalladas

T0. Congelar baseline, nombres y trazabilidad

Objetivo	Eliminar ambigüedades antes de implementar: versiones exactas, rutas, nombres de proyecto y artefactos reproducibles.
Depende de	Ninguna. Es la tarea que debe abrir el trabajo práctico.
Salidas esperadas	Repositorio de trabajo, fichero de baseline, árbol de directorios, tabla de evidencias y lista cerrada de

	versiones.
Criterio de cierre	Existe un documento baseline único y todo artefacto futuro referencia ese baseline por nombre y fecha.

Por qué existe esta tarea

Si no congelas versiones ahora, cualquier problema posterior será imposible de atribuir. En este TFG eso es especialmente crítico porque Vivado/Vitis están fijados a 2022.2, mientras que Buildroot debe concretarse a una release real y no a una etiqueta ambigua. [UG973] [BUILDROOT]

Pasos detallados

1. Crear una carpeta raíz del proyecto distinta de la memoria. Recomendación: `tfg\_pspl\_axi\_lite/`.
2. Dentro de esa raíz, crear al menos: `docs/`, `hw/vivado/`, `hw/ip/`, `sw/vitis/`, `sw/buildroot/`, `sw/linux-tests/`, `measurements/`, `logs/`, `artifacts/`.
3. Crear `docs/baseline.md` con: placa, versión exacta de Vivado, versión exacta de Vitis, versión real de Ubuntu, release exacta de Buildroot, nombre del proyecto Vivado, nombre del IP, política de nombres de ficheros y estrategia de arranque.
4. Añadir en el baseline el criterio de versionado de artefactos: por ejemplo `YYYYMMDD-hhmm` o hash de commit.
5. Crear un fichero `docs/evidence-matrix.md` con columnas: artefacto, etapa, herramienta, fecha, hash/firma, uso futuro en memoria.
6. Comprobar que el host corresponde a Ubuntu 20.04 LTS o una variante admitida por Vivado 2022.2. Si no coincide, registrar la desviación explícitamente. [UG973]
7. Anotar la duda de Buildroot como una decisión pendiente obligatoria: `Buildroot 2023.02.x exacto = \_\_\_\_\_`.

Riesgos y errores frecuentes

- Empezar Vivado antes de cerrar la release exacta de Buildroot.
- Guardar el proyecto Vivado dentro de directorios temporales o con nombres genéricos como `project\_1`.
- No registrar si el BOOT.bin y la SD se generaron con artefactos viejos o nuevos.

Evidencias que debes guardar

- Captura o nota de las versiones de Vivado/Vitis/Ubuntu.
- Fichero `baseline.md` versionado.
- Árbol de directorios inicial.
- Primera versión de `evidence-matrix.md`.

Nota de alcance

No pases a Vivado sin haber cerrado la nomenclatura del proyecto. Un error muy común en TFGs es tener varios `xpr`, varios `BOOT.bin` y varios `system.dtb` sin trazabilidad.

T1. Preparar el entorno host y confirmar soporte de herramientas

Objetivo	Verificar que el host es estable, que Vivado/Vitis arrancan y que no hay problemas de permisos, PATH o cable USB/JTAG.
-----------------	--

Depende de	T0 completada.
Salidas esperadas	Entorno host funcional, sin incertidumbres básicas de instalación.
Criterio de cierre	Vivado y Vitis abren correctamente, el host dispone de espacio suficiente y los comandos básicos de Buildroot funcionan.

Por qué existe esta tarea

No es una tarea burocrática: si el entorno falla aquí, perderás horas más adelante diagnosticando errores que no pertenecen al diseño. AMD documenta soporte de Vivado 2022.2 para Ubuntu 20.04 LTS y subversiones 20.04.1-20.04.4. [UG973]

Pasos detallados

- Comprobar `lsb_release -a`, `uname -a` y espacio libre en disco. Registrar el resultado en `docs/baseline.md`.
- Abrir Vivado 2022.2 y crear un proyecto vacío solo para verificar licencia/instalación.
- Abrir Vitis 2022.2 y confirmar que puede arrancar un workspace nuevo.
- Verificar que los binarios de Vivado/Vitis están en PATH o que existe un script claro de carga de entorno.
- Conectar la ZedBoard por USB-UART y, si procede, por JTAG. Confirmar que el host detecta los dispositivos y que puedes abrir el puerto serie.
- Preparar una carpeta de logs de consola serie (`logs/uart/`) desde el primer día.
- Instalar dependencias mínimas para Buildroot en el host; al menos `make`, `gcc`, `g++`, `bash`, `sed`, `patch`, `gzip`, `bzip2`, `perl`, `tar`, `wget` y la librería de `menuconfig` si vas a usarla. [BUILDROOT]

Riesgos y errores frecuentes

- Ignorar un fallo de puerto serie y seguir con el flujo; la UART es la herramienta más importante durante el arranque.
- Mezclar varias instalaciones de Vivado/Vitis sin documentar cuál se está usando.
- Usar Buildroot como root. La propia documentación recomienda construir como usuario normal. [BUILDROOT]

Evidencias que debes guardar

- Nota de sistema operativo real.
- Registro de detección de UART/JTAG.
- Lista de dependencias del host.

Nota de alcance

Si tu Ubuntu real no es 20.04 LTS, no bloquea automáticamente el TFG, pero debes anotarlo como desviación metodológica.

T2. Especificar el periférico AXI4-Lite antes de escribir RTL

Objetivo	Cerrar el contrato HW/SW del periférico antes de implementar lógica o tocar el device tree.
Depende de	T0 y T1.

Salidas esperadas	Documento corto de especificación, mapa de registros, anchos de bus y semántica de lectura/escritura.
Criterio de cierre	No quedan offsets ambiguos, todos los registros tienen reset value y la aplicación Linux sabe qué esperar en cada uno.

Por qué existe esta tarea

En tu propio TFG ya aparece una idea importante: la calidad de un periférico no depende solo de la lógica interna, sino de la claridad del contrato hardware-software. Si el mapa de registros queda ambiguo, la validación desde Linux deja de ser interpretable. [TFG-20-04]

Mapa de registros recomendado para el MVP

Offset	Nombre	Tipo	Uso recomendado
0x00	REG_ID	RO	Constante de identificación. Debe permitir detectar que el mapeo Linux está apuntando al periférico correcto.
0x04	REG_VERSION	RO	Versión del IP. Recomendada para comprobar coherencia entre bitstream y software.
0x08	REG_CONTROL	RW	Bits de control mínimos: limpiar contadores, reset lógico local o habilitar modo loopback.
0x0C	REG_STATUS	RO	Estado del periférico: listo, loopback activo, flags internas si las hubiera.
0x10	REG_WDATA	RW	Dato de prueba escrito desde Linux.
0x14	REG_RDATA	RO	Dato de retorno o eco; útil para verificar coherencia de lectura.
0x18	REG_WRITE_COUNT	RO	Contador de escrituras válidas sobre registros

Offset	Nombre	Tipo	Uso recomendado
			definidos.
0x1C	REG_READ_COUNT	RO	Contador de lecturas válidas sobre registros definidos.
0x20	REG_COUNTER_L	RO	Contador libre de baja palabra para observabilidad básica.
0x24	REG_COUNTER_H	RO	Alta palabra del contador si decides usar 64 bits.

Decisiones técnicas que debes escribir explícitamente

- Ancho de datos del periférico: 32 bits.
- Acceso alineado a palabra de 32 bits; nada de offsets impares o semánticas implícitas.
- Valores de reset de cada registro.
- Qué registros son RW, cuáles son RO y qué efectos laterales provoca cada lectura o escritura.
- Tamaño de la ventana de mapeo desde el punto de vista de software. Recomendación: una página o una ventana limpia documentada por Vivado; si el Address Editor asigna una región mayor, reflejar esa región tal cual en DTS.
- Política de errores: para offsets no implementados, decide si devuelves cero o dejas el comportamiento definido por la plantilla AXI. Lo importante es documentarlo y no cambiarlo después en silencio.

Pasos detallados

15. Crear ``hw/ip/axi_regbench/spec.md`` con propósito del IP, señales visibles, registros y semántica.
16. Definir un identificador constante para ``REG_ID`` y una codificación simple para ``REG_VERSION``.
17. Definir si ``REG_RDATA`` devuelve eco exacto de ``REG_WDATA`` o una transformación conocida. Para un MVP, el eco exacto es la opción más limpia.
18. Decidir si los contadores incrementan en toda lectura/escritura válida o solo en registros concretos. Recomendación: contar accesos a los registros de test y dejarlo explícito.
19. Generar también un header de software (``axi_regbench_regs.h``) con offsets y máscaras. Aunque todavía no escribas la aplicación, ese fichero actúa como contrato entre Vivado y Linux.

Riesgos y errores frecuentes

- Diseñar primero en VHDL y describir después el mapa de registros. Debe hacerse al revés.
- Usar nombres genéricos (``slv_reg0``, ``slv_reg1``) en la documentación final. Internamente puede valer; externamente, no.
- No incluir un registro de identificación. Sin él, un error de dirección base en Linux puede parecer un problema de latencia cuando en realidad es un problema de mapeo.

Evidencias que debes guardar

- Especificación del periférico.
- Header de offsets para software.
- Tabla cerrada de registros con valores de reset.

Nota de alcance

No necesitas un IP complejo para medir latencia de acceso a registros. De hecho, un IP demasiado complejo empeora la interpretabilidad del TFG.

T3. Implementar el RTL y simular el periférico

Objetivo	Convertir la especificación del IP en un esclavo AXI4-Lite correcto y verificable.
Depende de	T2 completada.
Salidas esperadas	Código VHDL del IP, testbench, formas de onda y resultado de simulación sin errores funcionales.
Criterio de cierre	La simulación demuestra reset correcto, escritura/lectura coherente, respuesta estable y ausencia de bloqueo en secuencias simples y back-to-back.

Por qué existe esta tarea

La memoria del TFG insiste en la verificación multinivel: primero semántica del bus, después consistencia del periférico, después integración con Linux. Aquí todavía no estás midiendo; estás demostrando que el IP es fiable. [TFG-20-04]

Pasos detallados

20. Crear el IP como AXI4-Lite slave de 32 bits. La ruta de menor riesgo es partir de la plantilla generada por Vivado y encapsular la lógica de usuario con el mínimo número posible de cambios en la interfaz AXI generada automáticamente.
21. Implementar los registros definidos en T2 y conectar su lógica de lectura/escritura a las posiciones previstas.
22. Asegurarte de que todos los registros tienen reset síncrono o asíncrono coherente con la plantilla elegida.
23. Crear un testbench que cubra, como mínimo: reset, lectura de `REG\_ID`, escritura de `REG\_WDATA`, lectura de `REG\_RDATA`, lectura de contadores, accesos repetidos y un acceso a offset no usado.
24. Añadir un caso back-to-back: varias escrituras seguidas y varias lecturas seguidas sin huecos artificiales, para verificar que el periférico no introduce bloqueos triviales.
25. Guardar formas de onda y una nota corta con el resultado de cada caso de prueba.

Riesgos y errores frecuentes

- Modificar indiscriminadamente la lógica AXI autogenerada sin aislar qué parte pertenece al protocolo y qué parte a tu lógica de usuario.
- No simular un reset limpio.
- Dar por buena una lectura funcional sin verificar qué ocurre tras varias transacciones consecutivas.

Evidencias que debes guardar

- Código fuente VHDL.
- Testbench.
- Capturas o exportación de formas de onda.
- Breve informe de verificación funcional.

Nota de alcance

Si el IP no queda limpio en simulación, no sigas hacia el hardware. Saltarte esta tarea multiplica el tiempo de depuración posterior.

T4. Crear el proyecto en Vivado y construir el Block Design base

Objetivo	Montar el sistema Zynq mínimo que permita al PS acceder al IP en la PL con reloj y reset correctos.
Depende de	T3 completada.
Salidas esperadas	Proyecto Vivado con PS configurado, IP integrado, clock/reset conectados y diseño validado.
Criterio de cierre	El Block Design pasa `Validate Design`, el reloj del AXI está unificado y existe una asignación clara de direcciones.

Por qué existe esta tarea

Esta tarea materializa la parte de antecedentes donde la memoria explica el flujo IP Packager + IP Integrator + Address Editor. [TFG-20-04] Además, el uso del camino GP para control encaja con la división de puertos de Zynq-7000. [UG585]

Pasos detallados

26. Crear un proyecto Vivado nuevo con el part/board de tu ZedBoard. Evita nombres por defecto.
27. Añadir el repositorio local de IPs y comprobar que tu IP aparece con nombre y versión correctos.
28. Instanciar `Zynq7 Processing System` y ejecutar la automatización inicial para aplicar la configuración base de placa.
29. Entrar en la configuración del PS y habilitar el puerto maestro que va desde PS hacia PL (`M\_AXI\_GP0`) y, al menos, un reloj FCLK hacia la PL.
30. Instanciar `proc\_sys\_reset` para generar un reset limpio sincronizado con el reloj del AXI del periférico.
31. Instanciar tu IP AXI4-Lite. Si Vivado no conecta directamente el maestro del PS con el esclavo del IP, añadir un AXI Interconnect o SmartConnect.
32. Conectar: reloj AXI del IP al FCLK elegido, reset del IP al `peripheral\_aresetn`, interfaz AXI del IP al interconnect y el interconnect al puerto GP del PS.
33. Ejecutar `Validate Design` hasta que no queden errores.
34. En `Address Editor`, asignar una dirección base limpia y documentarla inmediatamente en el baseline. No uses una dirección provisional sin dejar rastro.
35. Generar el wrapper HDL del diseño.

Riesgos y errores frecuentes

- Habilitar más puertos AXI de los necesarios y perder simplicidad.
- No registrar la dirección base final.
- Conectar resets o clocks distintos entre el interconnect y el periférico sin una razón clara.

Evidencias que debes guardar

- Captura del Block Design.
- Pantallazo o nota del Address Editor.
- Listado de parámetros relevantes del PS.

Nota de alcance

No añadas interrupciones en esta fase salvo que el plan experimental ya las necesite. Para el MVP, son complejidad extra sin valor inmediato.

T5. Sintetizar, implementar, generar bitstream y exportar XSA

Objetivo	Obtener un hardware coherente y cerrarlo con los artefactos que usarán Vitis y la parte software.
Depende de	T4 completada.
Salidas esperadas	Bitstream, XSA, informes de utilización y timing.
Criterio de cierre	Existe bitstream válido, no hay fallos graves de timing y el XSA exportado corresponde exactamente al diseño usado.

Por qué existe esta tarea

Desde este punto, cualquier medición posterior depende del hardware real; por tanto, los informes de timing y utilización pasan a ser parte de la evidencia del TFG, no solo residuos de herramienta.

Pasos detallados

36. Ejecutar síntesis del diseño completo y revisar advertencias relacionadas con el IP, clocks y resets.
37. Ejecutar implementación. Prestar atención a slack negativo, rutas problemáticas y clocks inesperados.
38. Si incluyes ILA en algún momento, no lo hagas en la primera pasada salvo que el diseño no funcione; la ILA cambia utilización y timing.
39. Generar el bitstream y exportar el hardware a XSA, incluyendo el bitstream si el flujo concreto lo permite.
40. Guardar en `artifacts/hw/` el `.bit` o `.bin` generado, el `.xsa`, el `timing\_summary` y el `utilization\_report`.
41. Actualizar `docs/evidence-matrix.md` con nombres y hashes.

Riesgos y errores frecuentes

- Cambiar el diseño y seguir usando un XSA viejo en Vitis.
- No guardar el timing summary que corresponde exactamente al bitstream desplegado.
- Introducir un ILA demasiado pronto y contaminar la línea base.

Evidencias que debes guardar

- Bitstream final.
- XSA final.
- Informes de timing y utilización.
- Hash o firma de artefactos.

Nota de alcance

Si aparece un problema de timing real, simplifica primero el periférico o elimina lógica no esencial. No tapes un problema estructural con complejidad adicional.

T6. Generar FSBL y BOOT.bin en Vitis/Bootgen

Objetivo	Preparar la imagen de arranque que programe la PL y entregue el control a U-Boot.
Depende de	T5 completada y decisión tomada sobre el U-Boot que se usará desde Buildroot.
Salidas esperadas	FSBL, fichero BIF o configuración equivalente y BOOT.bin reproducible.
Criterio de cierre	El sistema arranca desde SD usando el FSBL correcto, programa la PL y transfiere el control al segundo stage bootloader.

Por qué existe esta tarea

UG821 deja claro que el FSBL inicializa el PS, puede programar el bitstream y después carga la siguiente partición; además, el orden en la imagen de arranque importa: FSBL primero, bitstream a continuación y la aplicación/segundo stage después. [UG821]

Pasos detallados

42. Crear en Vitis un workspace limpio separado del proyecto Vivado.
43. Importar el XSA final y generar una plataforma para Zynq-7000.
44. Crear el proyecto FSBL y compilarlo sin modificaciones salvo que exista una razón muy clara.
45. Obtener `u-boot.elf` o artefacto equivalente desde la build de Buildroot que vayas a congelar. No uses un U-Boot suelto generado con otra configuración.
46. Construir el `BOOT.bin` con una descripción explícita del orden de particiones. Recomendación: conservar el `.bif` en el repositorio, aunque generes la imagen con asistente gráfico.
47. Estructura mínima recomendada del BIF: FSBL como bootloader, bitstream del sistema, U-Boot como segunda partición.
48. Guardar `BOOT.bin`, `fsbl.elf`, `.bif` y notas de versión en `artifacts/boot/`.

Riesgos y errores frecuentes

- Generar un `BOOT.bin` con un bitstream desalineado respecto al XSA.
- No conservar el fichero BIF o el método exacto con el que se empaquetó la imagen.
- Probar varias combinaciones de FSBL/U-Boot sin documentarlas.

Evidencias que debes guardar

- FSBL compilado.
- BIF o equivalente.
- BOOT.bin final.
- Registro de qué U-Boot se empaquetó dentro del BOOT.bin.

Nota de alcance

En este TFG, el bitstream no es opcional dentro del BOOT.bin: el periférico vive en la PL y debe cargarse para que Linux pueda verlo.

T7. Preparar Buildroot de forma reproducible

Objetivo	Construir un Linux embebido ligero y reproducible
----------	---

	para ZedBoard, manteniendo las customizaciones del proyecto separadas del árbol de Buildroot.
Depende de	T0, T1 y decisión sobre la release exacta de Buildroot.
Salidas esperadas	Defconfig guardado, árbol de customización, imágenes de U-Boot/kernel/rootfs y directorio de salida reproducible.
Criterio de cierre	Puedes reconstruir desde cero y obtienes el mismo conjunto de artefactos de salida y la misma configuración guardada.

Por qué existe esta tarea

Buildroot no es solo una forma rápida de sacar una rootfs: es la pieza que te permite fijar kernel, bootloader, device tree, overlay y utilidades de usuario en una misma cadena reproducible. El manual recomienda una estructura de customización específica, guardar la configuración con `savedefconfig` y no usar `output/target/` como sistema final desplegado. [BUILDROOT]

Decisión crítica previa

Congelar la release exacta

La etiqueta "2023.2" debe sustituirse por una release real del formato YYYY.MM. Si tu trabajo práctico se hará sobre un árbol 2023.02.x, anótalo exactamente así y conserva el tarball, commit o paquete descargado. [BUILDROOT]

Estructura recomendada de customización

```
sw/buildroot/
buildroot-2023.02.x/    # árbol upstream congelado
output-zed/             # salida out-of-tree
board/uco/zed-pspl/     # customizaciones de proyecto
linux.config
uboot.config
zed-pspl.dts / zed-pspl.dtsi
rootfs-overlay/
post_build.sh
post_image.sh
genimage.cfg            # si decides generar imagen SD automática
users_table.txt         # solo si hace falta
```

Variables y defines que conviene fijar desde el principio

Elemento a congelar	Uso en el proyecto	Observación práctica
Release Buildroot	Árbol base y compatibilidad global.	Guardar release exacta y no moverla a mitad del TFG.

Elemento a congelar	Uso en el proyecto	Observación práctica
`BR2_DEFCONFIG`	Ruta del defconfig guardado.	Usar `make savedefconfig` para congelarla. [BUILDDROOT]
`BR2_ROOTFS_OVERLAY`	Añadir scripts, binarios y ficheros del proyecto a la rootfs.	No meter cambios manuales directamente en `target/`.
`BR2_ROOTFS_POST_BUILD_SCRIPT`	Ajustes tras construir el target.	Úsalo solo para transformaciones finales necesarias.
`BR2_ROOTFS_POST_IMAGE_SCRIPT`	Empaquetado final, por ejemplo imagen SD.	Muy útil si acabas automatizando la SD.
Kernel config	Fijar soporte de Zynq, consola serie y opciones necesarias.	Guardar config o defconfig propio.
U-Boot config	Fijar board config y estrategia de arranque.	Guardar config o defconfig propio.
Formato rootfs	ext4/tar/cpio u otro.	Elige el mínimo necesario para tu boot real.
Paquetes de usuario	Herramientas mínimas de prueba.	Prioriza BusyBox, shell y utilidades básicas; Python en target solo si de verdad hace falta.

Pasos detallados

49. Descargar o clonar la release exacta de Buildroot y registrar su identificador en el baseline.
50. Trabajar siempre con build out-of-tree (`O=...`) para separar fuentes y salida. Buildroot lo soporta explícitamente y facilita builds paralelas limpias. [BUILDDROOT]
51. Crear la carpeta de customización del proyecto fuera del núcleo de Buildroot o, como mínimo, dentro de un árbol de proyecto claramente aislado. La documentación recomienda mantener customizaciones fuera del árbol principal o en una estructura de proyecto específica. [BUILDDROOT]
52. Arrancar desde una configuración cercana a Zynq/Zed si tu árbol concreto la ofrece; si no, montar una configuración ARM Cortex-A9 equivalente desde `menuconfig`.
53. Configurar arquitectura ARM, Cortex-A9, NEON/VFP según corresponda a la ZedBoard y a tu kernel/U-Boot elegidos.
54. Habilitar U-Boot, kernel Linux y la generación de las imágenes que realmente vas a desplegar.
55. Activar la consola serie adecuada y las utilidades mínimas para depurar: shell, `dmesg`, `hexdump`, `devmem` si está disponible, herramientas de montaje y utilidades básicas de BusyBox.
56. Definir `BR2\_ROOTFS\_OVERLAY` para poder instalar posteriormente scripts de campaña y, si procede, tu binario de pruebas.
57. Guardar la configuración con `make savedefconfig BR2\_DEFCONFIG=<ruta>`; registrar también cualquier config específica de U-Boot o kernel. [BUILDDROOT]
58. Construir con `make` y comprobar qué aparece en `output/images/`. Recuerda: `images/` contiene lo que debes desplegar; `target/` no es para copiarlo tal cual a la placa. [BUILDDROOT]

Riesgos y errores frecuentes

- Editar directamente el árbol upstream de Buildroot y perder el control de qué es customización y qué es origen.
- No guardar el defconfig y depender de un `.config` efímero.
- Copiar `output/target/` a la SD como si fuese la rootfs final. Buildroot indica que `target/` no tiene permisos ni nodos correctos para desplegarse así. [BUILDROOT]
- Instalar Python en target por costumbre. Para el TFG, la orquestación puede hacerse con shell y el análisis posterior en host.

Evidencias que debes guardar

- Release exacta de Buildroot.
- Defconfig guardado.
- Configs de kernel y U-Boot si las separas.
- Listado de imágenes generadas en `images/`.

Nota de alcance

La medida de latencia no exige una distribución compleja. Un sistema Buildroot pequeño y estable es preferible a una imagen más pesada con componentes que no aportan valor experimental.

T8. Integrar el device tree y el rootfs overlay del proyecto

Objetivo	Reflejar en software la existencia real del periférico y añadir a la rootfs los ficheros que usarás en validación.
Depende de	T4, T5 y T7 completadas.
Salidas esperadas	DTS/DTB del proyecto, nodo del periférico con dirección correcta y overlay de rootfs preparado.
Criterio de cierre	El DTB que realmente se arranca contiene el nodo del periférico con la dirección base correcta y la rootfs incorpora los ficheros necesarios de prueba.

Por qué existe esta tarea

La memoria ya subraya que cualquier incoherencia entre Vivado, device tree y software invalida la interpretación experimental. Esta tarea es el nexo explícito entre hardware y Linux. [TFG-20-04]

Pasos detallados

59. Decidir si vas a mantener un `.dts` completo o un `.dtsi` complementario. La clave no es el formato, sino que la personalización esté versionada y sea la que realmente compila Buildroot.
60. Añadir un nodo del periférico con `compatible` propio, `reg` con la dirección base y tamaño exactos del Address Editor, y `status = "okay"`.
61. No añadas interrupciones al DTS mientras el hardware no las tenga realmente cableadas y probadas.
62. Crear `rootfs-overlay/` y reservar dentro de ella al menos un lugar para los scripts de prueba y la configuración auxiliar del proyecto.
63. Si vas a usar inicialmente `/dev/mem`, el nodo del device tree sigue siendo útil porque documenta el hardware visible para Linux y mantiene la trazabilidad de la dirección base.

64. Compilar de nuevo Buildroot y verificar que el DTB generado corresponde a tu fuente modificada; evita la situación típica de editar un DTS y bootear otro distinto.

Riesgos y errores frecuentes

- Poner en el DTS una dirección base distinta a la del Address Editor.
- Editar un DTS local y arrancar un DTB viejo copiado en la SD.
- Usar un `compatible` genérico sin política clara; mejor un string propio del proyecto.

Evidencias que debes guardar

- Fuente DTS/DTSI final.
- DTB compilado realmente usado en arranque.
- Registro del `compatible`, `reg` y tamaño asignados.
- Contenido del rootfs overlay.

Nota de alcance

Recomendación de nodo mínimo:

```
axi_regbench@43c00000 {  
    compatible = "uco,axi-regbench-1.0";  
    reg = <0x43c00000 0x1000>;  
    status = "okay";  
};
```

Ajusta dirección y tamaño a tu caso real.

T9. Preparar la tarjeta SD, arrancar Linux y hacer la validación de humo

Objetivo	Pasar del entorno de desarrollo al sistema ejecutándose en la ZedBoard.
Depende de	T6, T7 y T8 completadas.
Salidas esperadas	SD funcional, arranque por serie documentado y primera validación de acceso a registros.
Criterio de cierre	La placa arranca de forma estable, llega a consola Linux y puedes leer al menos el registro de identificación del periférico.

Por qué existe esta tarea

Antes de cualquier campaña de medida, necesitas demostrar que el arranque completo funciona: BootROM -> FSBL -> bitstream -> U-Boot -> kernel -> rootfs -> acceso desde usuario. [UG821]

Pasos detallados

65. Elegir la estrategia de SD: partición de arranque + partición rootfs, o imagen SD completa si luego automatizas con `genimage`.
66. Copiar a la SD los artefactos correctos: `BOOT.bin` y el conjunto kernel/DTB/rootfs que corresponda exactamente a la build congelada.
67. Configurar o revisar `bootargs` y `bootcmd` en U-Boot. Plantilla mínima típica: `console=ttyPS0,115200 root=/dev/mmcblk0p2 rw rootwait` (ajustar a tu esquema real).

68. Arrancar con consola serie conectada y guardar el log completo desde el primer arranque funcional.
69. Ya en Linux, comprobar: fecha de build, memoria, consola, presencia del nodo en `/proc/device-tree`, y mensajes relevantes de kernel con `dmesg`.
70. Hacer una prueba de humo con `devmem` si la utilidad está disponible o con una herramienta mínima propia: leer `REG_ID`, escribir `REG_WDATA`, leer `REG_RDATA` y verificar el valor esperado.
71. Si falla la lectura del primer registro, no sigas hacia la medida; vuelve a revisar dirección base, DTB cargado, bitstream desplegado y orden de artefactos en SD.

Riesgos y errores frecuentes

- Bootear una SD con archivos mezclados de builds distintas.
- No capturar el log serie del arranque bueno y luego no poder demostrar qué kernel/DTB estabas usando.
- Interpretar un fallo de lectura como problema de hardware cuando puede ser simplemente un DTB antiguo o una base address incorrecta.

Evidencias que debes guardar

- Log UART del primer arranque funcional.
- Captura del primer `devmem` o prueba equivalente satisfactoria.
- Listado de archivos exactos copiados a la SD.

Nota de alcance

No avances hasta tener una lectura funcional inequívoca del periférico. Ese es el verdadero “hello world” del TFG práctico.

T10. Desarrollar la utilidad de usuario para acceso y medida con `/dev/mem`

Objetivo	Crear el software mínimo de usuario que permita validar y medir accesos de forma repetible.
Depende de	T9 completada.
Salidas esperadas	Binario de usuario en C, opciones de línea de comandos y salida en CSV o texto estructurado.
Criterio de cierre	La utilidad abre <code>/dev/mem</code> , mapea la región correcta, lee/escribe registros fiables y puede ejecutar bucles con temporización.

Por qué existe esta tarea

Para este TFG, el acceso más directo y controlable es `/dev/mem` + `mmap`. La propia memoria lo contempla como mecanismo válido para campañas controladas desde espacio de usuario. [TFG-20-04] Además, Buildroot no instala compilador en target por defecto, así que conviene compilar este binario en host o integrarlo como paquete reproducible. [BUILDROOT]

Pasos detallados

72. Crear en `sw/linux-tests/` una utilidad en C, no en shell, para el bucle interno de lectura/escritura. La shell puede orquestar, pero no debe cronometrar cada acceso individual.
73. Abrir `/dev/mem` con `O_RDWR | O_SYNC`, calcular la base alineada a página y mapear la ventana necesaria con `mmap`.
74. Implementar helpers `read32(offset)` y `write32(offset, value)` sobre punteros `volatile` de 32 bits.

75. Añadir un modo `smoke` que: lea `REG\_ID`, lea `REG\_VERSION`, escriba `REG\_WDATA`, lea `REG\_RDATA` y compruebe coherencia.
76. Añadir modos de campaña: `read\_loop`, `write\_loop`, `rw\_loop`, con número de iteraciones y warm-up configurables.
77. Añadir salida a CSV con campos mínimos: timestamp, build_id, base_addr, op, offset, iters, elapsed_ns, avg_ns, mismatches.
78. Definir códigos de retorno no nulos para error de apertura, error de `mmap`, error de ID y error de verificación funcional.
79. Decidir desde ya cómo se desplegará la utilidad en el target: copia manual inicial, overlay de rootfs o paquete local de Buildroot.

Riesgos y errores frecuentes

- Medir latencia con scripts shell puros.
- No comprobar antes el `REG\_ID` y empezar a medir una región errónea.
- Usar una base address hardcodeda en el binario sin reflejarla también en el baseline y en el DTS.

Evidencias que debes guardar

- Código fuente del binario de prueba.
- Cabecera de offsets incluida en el software.
- Primera ejecución correcta de modo `smoke`.

Nota de alcance

Recomendación metodológica: el binario de medida debe ser pequeño, determinista y sin dependencias innecesarias. El análisis estadístico vendrá después.

T11. Automatizar campañas y estructurar la captura de resultados

Objetivo	Pasar de una prueba manual a una campaña repetible y exportable.
Depende de	T10 completada.
Salidas esperadas	Scripts de campaña, directorios de resultados y metadatos asociados a cada ejecución.
Criterio de cierre	Cada campaña genera resultados estructurados, con nombre único y metadatos suficientes para repetirla o compararla.

Por qué existe esta tarea

La petición original del TFG ya pedía automatización y recolección de datos. Aquí debe resolverse sin inflar el sistema embebido: el núcleo de medida queda en C, y la automatización puede ser Bash ligera.

Pasos detallados

80. Crear `run\_smoke.sh` para validación rápida y `run\_benchmark.sh` para campañas completas.
81. Hacer que cada ejecución cree una carpeta nueva bajo `measurements/raw/` con fecha, hora y `build\_id`.
82. Guardar junto al CSV un fichero de metadatos con: versión del bitstream/XSA, versión del DTB, release Buildroot, clock FCLK, nombre del ejecutable y parámetros usados.

83. Implementar en el script el barrido de operaciones y de número de iteraciones. Recomendación mínima: lectura, escritura y secuencia lectura-escritura, cada una con varios tamaños de muestra.
84. Separar claramente `warmup` y `measurement`: el script debe lanzar una fase de calentamiento antes de registrar CSV.
85. Añadir verificación funcional previa al benchmark completo; si falla, abortar.
86. Guardar también el log de consola de la sesión o, como mínimo, un resumen de `dmesg` relevante.

Riesgos y errores frecuentes

- Sobrescribir ficheros de resultados por usar nombres genéricos como `results.csv`.
- No registrar qué bitstream/DTB se midió.
- Mezclar validación funcional y benchmark en un mismo script sin puntos de corte claros.

Evidencias que debes guardar

- Scripts de campaña.
- Estructura de carpetas de resultados.
- Ejemplo de CSV y ejemplo de metadatos.

Nota de alcance

Aunque en la propuesta original aparecían Python o Bash, para el target Buildroot conviene que Python no sea obligatorio. Bash + binario C es suficiente para esta fase.

T12. Fijar el protocolo experimental antes de ejecutar medidas masivas

Objetivo	Definir qué se va a medir, bajo qué condiciones y con qué repetición.
Depende de	T11 completada.
Salidas esperadas	Matriz experimental cerrada y condiciones de medida congeladas.
Criterio de cierre	Existe un plan de experimento escrito y ninguna campaña se ejecuta "a ojo".

Por qué existe esta tarea

Tu memoria ya dedica espacio a la necesidad de fijar iteraciones, warm-up, condiciones y tratamiento estadístico. Si no lo haces ahora, más tarde no sabrás qué resultados son comparables. [TFG-20-04]

Pasos detallados

87. Definir variables independientes: tipo de operación (R/W/RW), número de iteraciones, warm-up, número de repeticiones, estado de carga del sistema.
88. Definir variables dependientes: tiempo total, latencia media por acceso, jitter o desviación simple, mismatches funcionales y contadores internos del periférico si los usas como observabilidad.
89. Recomendación inicial: 10 repeticiones por punto experimental, con warm-up previo y, al menos, tres tamaños de campaña (por ejemplo 1e3, 1e4 y 1e5 accesos). Ajusta si el tiempo de ejecución lo exige.
90. Decidir si vas a medir solo sistema en reposo o también una condición con carga concurrente ligera. Si haces ambas, etiquétalas de forma separada y no mezcles los CSV.
91. Registrar frecuencia del reloj PL y cualquier mecanismo de escalado de CPU que pueda influir; si no puedes fijarlo, anota la limitación.

92. Cerrar por escrito qué resultados serán “propios” del TFG y qué comparaciones quedarán solo como apoyo bibliográfico.

Riesgos y errores frecuentes

- Cambiar el número de iteraciones a mitad de campaña sin dejar constancia.
- Comparar resultados de sistemas distintos como si fuesen equivalentes.
- Publicar throughput derivado de AXI4-Lite sin explicar cómo se ha calculado y cuál era el patrón de acceso.

Evidencias que debes guardar

- Matriz experimental documentada.
- Versión congelada del protocolo de medida.
- Lista de variables controladas y no controladas.

Nota de alcance

La honestidad metodológica es parte del valor del TFG: si algo no se pudo fijar, se dice explícitamente y se mide igualmente bajo esa limitación.

T13. Gestionar evidencias, hashes y trazabilidad para la futura memoria

Objetivo	Asegurar que cada resultado puede reconstruirse y relacionarse con un hardware/software concreto.
Depende de	T0-T12 en marcha.
Salidas esperadas	Manifest de artefactos, hashes, correspondencia entre resultados y builds, y material reutilizable para los capítulos futuros.
Criterio de cierre	Puedes responder qué hardware, qué DTB, qué binario y qué parámetros generaron cualquier CSV concreto.

Por qué existe esta tarea

Este TFG no se defiende solo con que “funcione”; se defiende con trazabilidad. La memoria actual insiste precisamente en eso: resultados trazables y reutilizables. [TFG-20-04]

Pasos detallados

93. Crear un `manifest.yml` o `manifest.md` por campaña con: bitstream hash, XSA hash, FSBL, BOOT.bin, release Buildroot, defconfig, DTB, nombre del ejecutable, commit o versión y fecha.
94. Guardar en `artifacts/` una copia cerrada de los ficheros clave usados en la campaña o, si son grandes, su hash y ruta inmutable.
95. Añadir una tabla de correspondencia entre artefactos y capítulos futuros del TFG: diseño HW, integración Linux, metodología, resultados.
96. Reservar en `docs/` una carpeta con capturas reutilizables: Block Design, Address Editor, consola de arranque, prueba de humo, estructura de resultados.
97. No borrar logs "temporales" que permitan explicar un problema resuelto; pueden ser útiles en discusión y riesgos.

Riesgos y errores frecuentes

- Quedarte solo con los CSV finales y perder los metadatos.
- No distinguir qué DTB o qué BOOT.bin estaba realmente en la SD en cada momento.
- No guardar las capturas de la etapa de integración cuando ya todo funciona.

Evidencias que debes guardar

- Manifest por campaña.
- Tabla de correspondencia artefacto-capítulo.
- Carpeta de capturas y logs reutilizable para la memoria.

Nota de alcance

Esta tarea parece documental, pero en realidad reduce muchísimo el tiempo de redacción posterior y evita contradicciones entre lo implementado y lo escrito.

T14. Extensiones opcionales controladas: UIO, IRQ e ILA

Objetivo	Ampliar el sistema solo cuando el MVP esté cerrado y medido.
Depende de	T9-T13 cerradas para el MVP.
Salidas esperadas	Extensiones opcionales con justificación técnica clara y sin romper la línea base.
Criterio de cierre	Existe una razón concreta para cada extensión y una nueva línea base si se introduce.

Por qué existe esta tarea

La parte teórica del TFG menciona UIO, interrupciones e instrumentación como posibilidades, pero no son requisitos del caso de estudio mínimo. Deben tratarse como ampliaciones y no como condición de arranque.

Pasos detallados

98. UIO: si quieres un acceso de usuario más estructurado o explorar interrupciones sin driver completo, estudiar ``uio_pdrv_genirq`` como extensión. La documentación del kernel indica que, en sistemas con device tree, la sonda requiere alinear el ``compatible`` del nodo con el parámetro ``of_id``. [UIO]
99. Interrupciones: solo añadirlas si el periférico ya es funcional por polling y existe una hipótesis experimental clara que justifique medir la vía de evento.
100. ILA: introducirla solo para depuración puntual del handshake AXI o correlación de eventos. Guardar siempre una línea base sin ILA, porque cambia recursos y timing.
101. Si realizas una ampliación, crear una rama o variante de configuración separada y no contaminar los resultados del MVP.

Riesgos y errores frecuentes

- Usar UIO antes de tener una prueba estable con ``/dev/mem``.
- Medir con ILA activa y luego comparar esos datos con una build sin ILA como si fuesen equivalentes.
- Ampliar a AXI HP/DMA/Stream sin haber cerrado el caso de estudio AXI4-Lite.

Evidencias que debes guardar

- Documento específico de la extensión.
- Nueva baseline o variante declarada.

- Justificación explícita de por qué la extensión aporta valor al TFG.

Nota de alcance

La extensión UIO es útil, pero no es el camino mínimo. En UIO, `/dev/uidoX` también se usa con `mmap`, y las interrupciones se gestionan leyendo del propio fichero de dispositivo. [UIO]

5. Secuencia recomendada de ejecución real

Orden	Bloque	Resultado que debes obtener antes de continuar
1	T0-T1	Entorno congelado y sin dudas básicas.
2	T2-T3	IP definido y verificado en simulación.
3	T4-T5	Block Design limpio, bitstream y XSA válidos.
4	T6	BOOT.bin reproducible.
5	T7-T8	Buildroot configurado, DTS integrado y rootfs preparada.
6	T9	Linux arranca y lee el primer registro.
7	T10-T11	Utilidad de usuario y campaña automatizada.
8	T12-T13	Protocolo experimental congelado y trazabilidad completa.
9	T14	Solo si queda tiempo y el MVP está completamente cerrado.

Regla práctica

No intentes diseñar el capítulo de resultados antes de cerrar T9. El primer hito real no es tener Linux, sino tener Linux leyendo el registro correcto del periférico correcto.

6. Errores que más probablemente te harán perder tiempo

- Confundir la versión real de Buildroot y hablar de “2023.2” sin fijar la release exacta.
- Modificar Vivado y Buildroot a la vez; si falla el arranque, no sabrás dónde está el problema.
- Bootear una SD con un `BOOT.bin` antiguo.
- Editar un DTS pero arrancar con otro DTB distinto.
- No registrar la dirección base asignada por el Address Editor.

- Medir desde shell y concluir sobre latencia a partir de tiempos dominados por el propio intérprete.
- Usar `output/target/` como si fuera una rootfs desplegable.
- Activar interrupciones, UIO o ILA antes de cerrar el camino mínimo.
- No guardar el log serie del primer arranque bueno.

7. Plantillas mínimas que conviene dejar creadas

7.1 Manifest de build/campaña

```
project: pspl-axi-lite-zed
date: 2026-04-20
board: ZedBoard
vivado: 2022.2
vitis: 2022.2
buildroot: 2023.02.x
xsa: artifacts/hw/system.xsa
bitstream_hash: <rellenar>
boot_bin_hash: <rellenar>
dtb: zed-pspl.dtb
base_addr: 0x43C00000
clock_pl: <rellenar>
test_binary: axi_regbench
protocol_version: v1
```

7.2 Header de offsets para software

```
#define AXI_REGBENCH_REG_ID      0x00
#define AXI_REGBENCH_REG_VERSION 0x04
#define AXI_REGBENCH_REG_CONTROL 0x08
#define AXI_REGBENCH_REG_STATUS  0x0C
#define AXI_REGBENCH_REG_WDATA   0x10
#define AXI_REGBENCH_REG_RDATA   0x14
#define AXI_REGBENCH_REG_WR_COUNT 0x18
#define AXI_REGBENCH_REG_RD_COUNT 0x1C
```

7.3 BIF mínimo orientativo

```
the_ROM_image:
{
  [bootloader] fsbl.elf
  system.bit
  u-boot.elf
}
```

8. Qué queda preparado para pedir después

Una vez ejecutada esta guía, el siguiente paso lógico ya no será “qué tengo que hacer”, sino desarrollar cada artefacto concreto: VHDL del periférico, DTS final, defconfig de Buildroot, binario de usuario y scripts de

campaña. Es decir: esta guía te deja el alcance práctico cerrado y listo para bajar a implementación de detalle.

En ese momento, sí tendrá sentido generar documentos de apoyo adicionales, por ejemplo: a) checklist de Vivado, b) checklist de Buildroot, c) plantilla del binario `axi\_regbench`, d) plantilla de campaña de medidas y e) mapa de trazabilidad hacia los capítulos de diseño, integración, metodología y resultados.

Fuentes operativas usadas para esta guía

- [TFG-20-04] Memoria de referencia actual: "TFG-JavierCarmonaTejero-20-04.pdf".
- [PETICION] Petición inicial del TFG: "PeticionTFG-JavierCarmonaTejero.pdf".
- [UG585] AMD, Zynq-7000 SoC Technical Reference Manual (UG585), especialmente el resumen de interfaces PS-PL y el papel de GP/HP/ACP.
- [UG821] AMD, Zynq-7000 SoC Software Developers Guide (UG821), especialmente arranque, FSBL, U-Boot y creación de imágenes de arranque.
- [UG973] AMD, Vivado 2022.2 Release Notes / Installation Guide (UG973), para soporte de sistema operativo en host.
- [BUILDROOT] Buildroot user manual, especialmente estructura recomendada, customizaciones fuera del árbol, savedefconfig y diferencia entre images/, target/ y host/.
- [UIO] Linux Kernel documentation, Userspace I/O HOWTO, solo como referencia opcional para la extensión UIO; no es requisito para el MVP de este TFG.
- [BIBLIO-RESUMEN] Resumen bibliográfico interno: "deep-research-report.md". Se ha usado para mantener coherencia con el marco académico, pero esta guía prioriza fuentes técnicas operativas.